

LipiSetu: Script-Invariant Hindi Search

CSD358 IR Hackathon · Track T5: Multilingual and Indic-language search

Team: Devansh Pandey (2310110461), Anamika Pal (2310110037), Abhinav Bachchas (2310110383) · Code: github.com/devansh101005/IR-Hackathon · Demo video: [\[unlisted link\]](#)

Most Hindi users type Hindi in Roman letters, each with their own spelling, while most Hindi text is written in Devanagari. A standard search engine therefore serves them far worse: on 350 judged MIRACL-Hindi queries, BM25 reaches nDCG@10 0.533 for Devanagari queries but only 0.004 for the same questions typed in Roman script, and the obvious fix (transliterating the query) only reaches 0.113. We call this loss the **Script Gap**. LipiSetu closes most of it with classic IR: a cross-script phonetic key (the **Dhvani key**) indexed as its own zone, **pooled document frequency** across spellings, and query-term proximity, all inside a BM25 engine written from scratch. A distilled, int8 neural query encoder is run only when a gate trained on IR signals predicts that it will help.

1. Problem and track relevance

User need. On phones, Hindi is mostly typed in Roman script ("kal ka mausam"), and the same word is spelled in many ways (mausam / mosam / mousam). The content people search (Wikipedia, news, government pages) is mostly in Devanagari ("कल का मौसम"). An inverted index matches exact terms, so these users silently get worse results. The problem is also inside the data itself: 19% of the MIRACL-Hindi passages we index already contain Roman-script words.

Why it belongs to T5. The track asks for search in an Indian language with code-mixed or transliterated queries. Our system touches every IR hook listed for T5: tokenisation and normalisation for Devanagari, a Hindi stemmer compared with no stemming, Soundex-style phonetic matching for transliterated spellings, the behaviour of stop words and idf on a Hindi corpus, and cross-lingual ranking in a vector space.

Papers referred. Mixed-script IR was studied by Gupta et al. (SIGIR 2014) and in the FIRE 2015 MSIR shared task; we use MIRACL (Zhang et al., TACL 2023) for judged Hindi queries. Pooled df follows Pirkola's structured queries (SIGIR 1998); the consistency metric is Rank-Biased Overlap (Webber et al., TOIS 2010); the distillation follows Reimers and Gurevych (EMNLP 2020); the gate follows query performance prediction (NQC, Shtok et al.) and cascade ranking (Wang, Lin and Metzler, SIGIR 2011). Full list in the references.

2. How we used IR

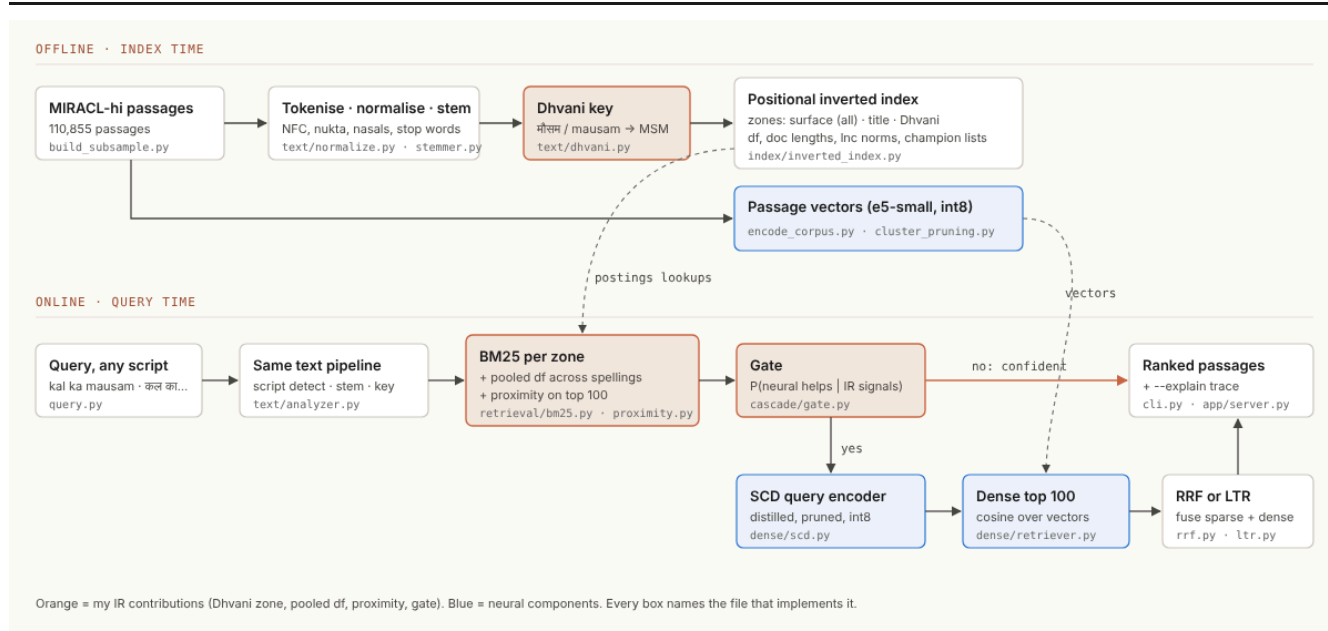


Figure 1. LipiSetu pipeline. Orange boxes are our IR contributions, blue boxes are neural components. Every box names the file that implements it.

What is a document. One MIRACL passage is one document (relevance labels are per passage), with zones *title* and *body*. We index every passage judged for a dev or train query plus a seeded random sample, 110,855 passages and 8,081,081 tokens in total (scripts/build_subsample.py).

Tokenisation and normalisation (text/tokenize.py, normalize.py). Python's \w does not match Devanagari vowel signs, so a regex tokenizer cuts हिन्दी into three pieces; we tokenise with explicit Unicode ranges instead. Normalisation applies NFC, folds nukta (क़→क), chandrabindu to anusvara, removes zero-width joiners, maps Devanagari digits, rewrites a half nasal before a consonant as an anusvara (हिन्दी = हिंदी), and case-folds Roman text.

Stop words and idf on Hindi (text/stopwords.py, scripts/corpus_stats.py). Hindi stop words are extremely frequent postpositions and auxiliaries: the eight highest-df terms are के, में, है, की, का, और, से, एक. 35 of the 40 highest-df terms are in our stop list (Hindi, Hinglish and English question words included). The corpus follows Zipf's law (appendix), and 151,995 of the 274,129 vocabulary terms occur only once, so most of the vocabulary has a very high idf. Stop words keep their positions so phrase queries keep the right gaps.

Stemming (text/stemmer.py). A light suffix stripper after Ramanathan and Rao (2003) maps लड़का / लड़के / लड़कों to one stem. It raises Devanagari nDCG@10 from 0.492 (no stemming, B1N) to 0.533 (B1).

Dhvani key (text/dhvani.py). A Soundex-style key that works on both scripts: every consonant sound maps to one class, aspirated = unaspirated, retroflex = dental, w = v, z = j, f = ph, a nasal before a consonant becomes N, vowels are dropped except at the start (which also handles Hindi's silent inherent vowel), and repeated classes collapse. Unlike English Soundex the key is not cut to four characters. मौसम, mausam, mosam and mousam all become MSM. Classic Soundex (text/soundex.py) is implemented as a comparison.

Inverted index with zones (index/inverted_index.py). A dictionary plus one postings file packed into numpy arrays, with positions. Zones: *all* (stemmed title + body), *title*, *dhvani* (keys of every word), plus *soundex*, *nostem* and *raw* zones for the baselines. Boolean AND/OR/NOT processes terms in order of increasing df and uses skip pointers; phrase queries use positions (retrieval/boolean.py).

Scoring (retrieval/bm25.py, tfidf.py, proximity.py, topk.py). BM25 per zone, combined with zone weights, accumulated term-at-a-time in one score slot per document, with heap-based top-K. tf-idf Inc.Itc cosine is implemented as the vector-space comparison.

```
score(d) = BM25all(d) + wdhvani · BM25dhvani(d) + wtitle · BM25title(d) + λ · (m-1)/(window-1)
```

The last term is **query-term proximity**: the smallest window of Dhvani positions that contains all m matched query words, applied to the top 100. **Pooled df**: a surface term uses the df of its Dhvani class, i.e. the number of documents containing any spelling of that word (Pirkola's structured query, applied to spelling variants instead of translations). All parameters were tuned on the **train** split only, with the average nDCG@10 over the three query scripts as the objective: k1 = 0.9, b = 0.5, w_{dhvani} = 2.0, w_{title} = 1.0, λ = 2.0. Champion lists (r = 200) are precomputed for the speed experiment.

3. Beyond IR

Dense retrieval (dense/). Passages are encoded once with multilingual-e5-small, exported to ONNX and quantised to int8 (cosine with the fp32 vectors ≈ 0.98). Search is a dot product over the passage vectors, or **cluster pruning** from the lectures (√N leaders; only the followers of the 8 nearest leaders are scored).

Script-Consistency Distillation (SCD). A student copy of the encoder is trained so that a Roman spelling of a train query lands on the teacher's embedding of the Devanagari query (loss = 1 – cosine). Only the query encoder changes, so the passage vectors are reused. On held-out train queries the cosine between a Roman query and its Devanagari target rises from 0.783 to 0.902. The 250,002-token vocabulary is then pruned to the 43,366 tokens used by the corpus and train queries, and the model is quantised: 448 MB → 37 MB.

Learning to rank (rerank/ltr.py). A logistic regression over IR features of each candidate (BM25 per zone, proximity, tf-idf, dense cosine, reciprocal ranks, a static quality prior g(d) for the first passage of an article, length). Trained on train judgments.

Gated cascade (cascade/gate.py). Features that the sparse stage gives for free (top score, score gap, NQC, share of words matched only through Dhvani, unmatched words, max idf, length, script) predict whether the

neural stage will improve nDCG@10. The threshold is chosen on train as the fewest neural calls that keep 99% of the always-neural quality.

4. Novelty and creativity

The obvious T5 project (and the PDF's own sample idea) is a Hinglish engine that transliterates the query and ranks with BM25; that is our baseline B2. What is different:

aspect	obvious baseline	LipiSetu
Evaluation	average nDCG / P@k only	Script Gap, cross-script consistency (RBO@10), worst-script nDCG
Roman queries	transliterate the query, then BM25 (B2)	Dhvani zone in the index; no transliteration model
df for variants	each spelling has its own df	pooled df across spellings (Pirkola-style)
Neural stage	always on, or absent	run only when an IR-signal gate predicts it helps
Model size	off-the-shelf encoder	script-consistency distillation + vocabulary pruning + int8
Ranking signals	fixed weights	zone weights tuned on train; learning to rank over IR features

Prior art, honestly. Mixed-script IR is not new (Gupta et al. 2014; FIRE MSIR) and Indic Soundex variants exist (e.g. libindic). We do not claim to be first. To our knowledge the combination here is new for Hindi on MIRACL: script invariance measured explicitly, a cross-script phonetic zone with pooled df inside a classic BM25 engine with a rule-by-rule ablation, and neural inference gated by IR signals and reported as a quality-versus-compute curve.

5. Evaluation

Setup. Test: the 350 MIRACL-Hindi dev queries (3,494 judgments, about 2.1 relevant passages per query, so P@10 cannot exceed about 0.2 and nDCG@10 is our main metric). Every query is run in three forms: F1 the original Devanagari, F2 a standard Hinglish romanisation, F3 a casual romanisation with seeded spelling noise (text/romanize.py). Tuning and training use only the 1,169 train queries. Unjudged passages count as non-relevant. **Script Gap** = nDCG(F1) - nDCG(form); **CSC@10** = mean pairwise RBO (p = 0.9) between the top-10 lists of the three forms; **worst-script nDCG** = mean over queries of the lowest nDCG@10 among the forms.

Table 1. Effectiveness (dev, 350 queries)

	system	nDCG@10 Deva	nDCG@10 Roman	nDCG@10 casual	P@5 Deva	R@100 Deva	R@100 casual
B0	BM25, raw whitespace tokens	0.395	0.000	0.000	0.173	0.785	0.006
B1	BM25 + normalisation + stemming	0.533	0.004	0.003	0.231	0.937	0.012
B1N	B1 without stemming	0.492	0.004	0.003	0.217	0.903	0.012
B2	B1 + Roman→Devanagari transliteration	0.531	0.113	0.095	0.230	0.937	0.334
V1	tf-idf Inc.ltc cosine	0.286	0.004	0.004	0.127	0.833	0.009
S0	B1 + classic Soundex zone	0.518	0.344	0.320	0.219	0.915	0.698
S1	B1 + Dhvani zone	0.544	0.481	0.488	0.239	0.933	0.865
S2	S1 + pooled df	0.533	0.487	0.490	0.234	0.926	0.865
S3	S2 + title zone + proximity	0.543	0.488	0.491	0.246	0.932	0.865
D0	dense e5-small (int8)	0.601	0.001	0.000	0.258	0.954	0.007
D1	dense, SCD query encoder	0.598	0.167	0.151	0.254	0.964	0.391
H1	RRF(S3, D1)	0.636	0.427	0.426	0.279	0.981	0.865
L1	learning to rank	0.642	0.501	0.498	0.288	0.993	0.895
G1	gated cascade	0.633	0.492	0.490	0.277	0.977	0.871

Table 2. Script invariance

system	Script Gap Roman	Script Gap casual	CSC@10 (RBO)	worst-script nDCG@10
B0 BM25, raw whitespace tokens	0.395	0.395	0.228	0.000
B1 BM25 + normalisation + stemming	0.529	0.530	0.179	0.002
B1N B1 without stemming	0.488	0.489	0.179	0.003
B2 B1 + Roman→Devanagari transliteration	0.418	0.436	0.244	0.066
V1 tf-idf Inc.ltc cosine	0.281	0.282	0.185	0.002
S0 B1 + classic Soundex zone	0.174	0.198	0.534	0.296
S1 B1 + Dhvani zone	0.063	0.057	0.774	0.452
S2 S1 + pooled df	0.046	0.043	0.794	0.456
S3 S2 + title zone + proximity	0.054	0.051	0.748	0.444
D0 dense e5-small (int8)	0.600	0.600	0.169	0.000
D1 dense, SCD query encoder	0.432	0.447	0.299	0.127
H1 RRF(S3, D1)	0.209	0.210	0.508	0.353
L1 learning to rank	0.141	0.143	0.637	0.456
G1 gated cascade	0.141	0.143	0.625	0.432

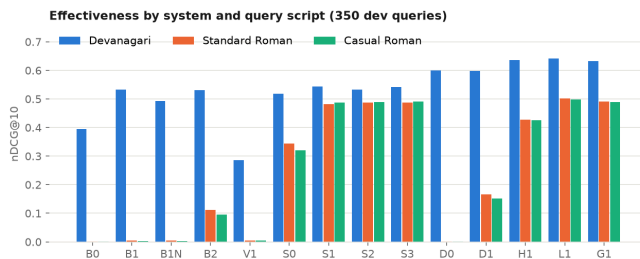


Figure 2. nDCG@10 by system and query script.

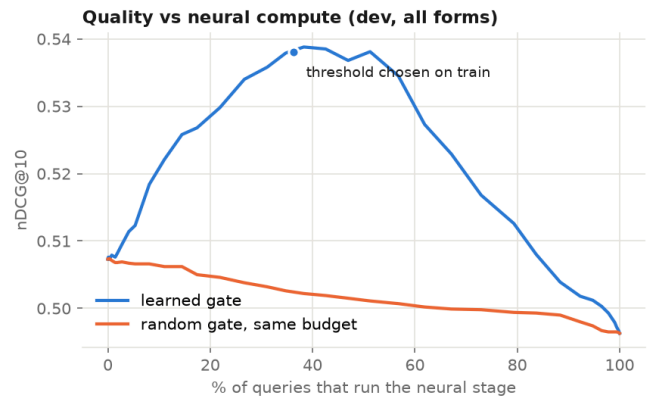


Figure 3. Quality vs share of queries that run the neural stage.

The Script Gap is real and large. BM25 with stemming (B1) reaches 0.533 on Devanagari queries and 0.004 on the same questions in Roman script. The obvious fix, transliterating the query (B2), only recovers 0.113 / 0.095 (standard / casual Roman), because casual Hinglish spellings are not valid transliteration input ("bharat" becomes भरत, not भारत). Classic Soundex (S0) helps (0.344) but its 4-character English codes merge too much. The Dhvani zone (S1) reaches 0.481 / 0.488, and with pooled df (S2) the casual-Roman Script Gap falls from 0.436 (B2) to 0.043, while cross-script consistency rises from 0.244 to 0.794. Every one of these Roman-query gains is significant ($p < 0.001$).

Neural models have their own script problem. The base dense model (D0) is the best single system for Devanagari (0.601) but scores 0.000 on casual Roman: for a Roman query it retrieves other Roman-script passages, i.e. it matches the script before the meaning. Script-consistency distillation (D1) fixes part of this (0.151) at almost no cost on Devanagari (0.598), but stays far below the Dhvani zone on Roman queries. Plain fusion (H1) is best on Devanagari (0.636) but drags Roman queries down to 0.427, so fusion alone is not script-invariant. Learning to rank (L1) uses both signals and is the best system on every script (0.642 / 0.501 / 0.498), with the highest worst-script nDCG (0.456 vs 0.066 for B2).

The gate spends neural compute where it helps. Because the neural stage helps Devanagari queries and hurts many Roman ones, always running it (0.496 over all dev queries and forms) is worse than never running it (0.507). The gate, with its threshold chosen on train, runs the neural stage for 36% of queries and reaches

0.538, better than both extremes and well above a random gate with the same budget (Figure 3; an oracle reaches 0.598 at 31%). The cascade's median latency is 18.6 ms against 23.8 ms for always-hybrid and 35.5 ms for LTR.

tf-idf vs BM25. Inc.ltc (V1) scores 0.286 against 0.533 for BM25. Full cosine normalisation over-rewards short passages: the median length of V1's top passage is 12 terms, against 35 for the corpus and 57 for BM25, whose tuned $b = 0.5$ normalises length only partly (results/e13_length_bias.json).

Pooled df. On the Devanagari-only corpus pooled df trades a little Devanagari accuracy (0.544 $\rightarrow$ 0.533) for Roman accuracy and consistency. On the mixed-script corpus, where a word's df really is split between spellings, it helps every form (Table 4): standard Roman 0.314 $\rightarrow$ 0.358, casual Roman 0.351 $\rightarrow$ 0.390, consistency 0.346 $\rightarrow$ 0.408.

Ablation. Dropping vowels is by far the most important Dhvani rule, followed by merging retroflex and dental sounds (Table 3). Collapsing doubled letters slightly hurts; we keep the rule set fixed as designed rather than tune it on the test queries.

Table 3. Dhvani rule ablation (one rule switched off, Dhvani zone rebuilt)

variant	keys	words/key	nDCG Deva	nDCG Roman	nDCG casual	CSC@10
all rules (Dhvani)	100,839	2.65	0.544	0.481	0.488	0.774
without merge_aspiration	111,942	2.39	0.547	0.494	0.479	0.760
without merge_retroflex	111,891	2.39	0.546	0.382	0.388	0.651
without drop_vowels	219,235	1.23	0.518	0.166	0.123	0.319
without merge_wv	101,689	2.63	0.544	0.481	0.480	0.763
without merge_zj	101,232	2.64	0.544	0.479	0.482	0.755
without merge_fp	101,841	2.62	0.544	0.480	0.486	0.771
without nasal_rule	101,007	2.64	0.544	0.472	0.478	0.767
without collapse_doubles	113,486	2.36	0.542	0.484	0.488	0.788

Table 4. Pooled df on a mixed-script corpus (30% of passages romanised, synthetic)

system		nDCG Deva	nDCG Roman	nDCG casual	word	key	df Deva	df Roman	df pooled	idf Deva	idf Roman	idf pooled
B1	BM25 + normalisation + stemming	0.416	0.201	0.150	यंत्र / yantra	YNTR	319	103	351	5.85	6.98	5.75
B2	B1 + Roman $\rightarrow$ Devanagari transliteration	0.415	0.106	0.089	समय / samay	SMY	6,555	2,700	9,260	2.83	3.71	2.48
S1	B1 + Dhvani zone	0.460	0.314	0.351	अधिकतम / adhikatham	ADKTM	331	145	492	5.81	6.64	5.42
S2	S1 + pooled df	0.465	0.358	0.390	अंतर्गत / antargat	ANTRGT	3,737	1,603	5,378	3.39	4.24	3.03
S3	S2 + title zone + proximity	0.464	0.332	0.375	स्थित / sthit	ST	9,047	3,165	18,994	2.51	3.56	1.76
					संख्या / sankhya	SNKY	2,036	807	2,918	4.00	4.92	3.64
					वृद्धि / vridhhi	VRD	957	385	3,641	4.75	5.66	3.42
					शक्ति / shakti	SKT	1,483	473	13,896	4.31	5.46	2.08

Table 5. Efficiency structures

experiment	variant	median ms	nDCG@10	comparisons	docs scored	overlap@100
top-K selection	heap over scored docs (mine)	5.957	–	–	–	–
top-K selection	full sort of all N scores	1.252	–	–	–	–
champion lists (S1, r=200)	full postings	7.713	0.5045	–	–	–
champion lists (S1, r=200)	champion lists	1.282	0.3739	–	–	–
AND merge (pairs of dev query terms)	plain merge	–	–	4606	–	–
AND merge (pairs of dev query terms)	skip pointers (sqrt L)	–	–	2354	–	–
dense search (D0)	exact (all N vectors)	5.62	0.2005	–	110855	–
dense search (D0)	cluster pruning (sqrt N leaders, 8 clusters)	6.663	0.1277	–	5284	0.5024

Skip pointers halve the comparisons of AND merges. Champion lists are 5-6x faster but lose a lot of quality, mostly for Roman queries whose Dhvani postings are long. Cluster pruning scores about 5% of the vectors and keeps half of the exact top 100, but on this laptop it is not faster: one numpy dot product over all 110k vectors already takes only a few milliseconds, so collecting the followers costs as much as it saves. It should only pay off on a much larger collection. In pure Python the heap top-K is slower than numpy's C sort even though it is asymptotically better.

Significance. Paired randomisation tests (10,000 permutations, nDCG@10, appendix D) give $p < 0.001$ for every Roman-query gain listed above (S1 over B1 and S0, S3 over B2, D1 over D0, L1 and G1 over H1) and for H1 over S3 on Devanagari. On Devanagari queries the sparse systems are not significantly different from B1/B2 (no loss), and G1 is not significantly different from the always-hybrid H1, even though it runs the neural stage for only a third of the queries.

Human query set. Three team members romanised the same 60 dev queries independently (no transliteration tools), and each wrote code-mixed and English versions of 20 of them. All three spelled a word identically for 78% of words, but the Dhvani keys agreed for 91%. On the real romanisations BM25 reaches only 0.013–0.020 and transliteration 0.112–0.140, while S3 reaches 0.435–0.448 ($p \leq 0.0002$ against B2 for every annotator) and learning to rank 0.491–0.507 ($p \leq 0.0389$ against H1). So the synthetic results hold for real spelling variation. Code-mixed and English queries are different: a phonetic key cannot link *leader* to नेता, so S3 drops to 0.204 (code-mixed) and 0.119 (English), and only the dense stage helps (D1 0.413 on English, L1 0.383 on code-mixed). The gate was trained on Devanagari and romanised queries only, and it under-calls the neural stage here: G1 is below the always-hybrid H1 on code-mixed (0.260 vs 0.341, $p = 0.0095$) and English (0.258 vs 0.333, $p = 0.0009$).

system	Deva	Roman A1	Roman A2	Roman A3	code-mixed	English
B1 BM25 + normalisation + stemming	0.571	0.020	0.019	0.013	0.033	0.015
B2 B1 + Roman→Devanagari transliteration	0.571	0.140	0.112	0.117	0.054	0.039
S0 B1 + classic Soundex zone	0.568	0.270	0.270	0.253	0.100	0.051
S3 S2 + title zone + proximity	0.616	0.448	0.435	0.438	0.204	0.119
D0 dense e5-small (int8)	0.611	0.005	0.005	0.005	0.043	0.346
D1 dense, SCD query encoder	0.595	0.188	0.208	0.179	0.316	0.413
H1 RRF(S3, D1)	0.697	0.437	0.404	0.433	0.341	0.333
L1 learning to rank	0.706	0.507	0.507	0.491	0.383	0.369
G1 gated cascade	0.691	0.454	0.438	0.434	0.260	0.258

6. Limitations and next steps

- The corpus is a subsample of MIRACL-Hindi (all judged passages + a random sample), so absolute scores are not comparable to the MIRACL leaderboard; unjudged passages count as non-relevant.

- The Roman query forms and the mixed-script corpus are synthetic (my romaniser); the human query set is the realistic check.
- Dhvani merges some different words: *kal* (कल, yesterday) and खेल (khel, game) share the key KL, because aspiration and vowels are dropped. Words whose spoken form drops a vowel (कमला / kamla) get different keys. Zone weights limit the damage but do not remove it.
- English words inside code-mixed queries are only matched by the dense stage, and the gate never saw code-mixed or English queries in training, so it runs the dense stage too rarely for them.
- The human set is small (60 queries), and one annotator (me) designed the Dhvani key, so that romanisation is not blind.
- Hindi only.

Course-project roadmap. (1) Bengali and Telugu from MIRACL with per-language Dhvani tables; (2) a larger human query set and voice queries; (3) the int8 encoder in the browser with ONNX Runtime Web, so search runs fully on the device; (4) learned key rules instead of hand-written ones; (5) a short paper for FIRE.

7. Work division

Devansh Pandey: idea and system design; text pipeline, inverted index, Boolean/phrase, tf-idf, BM25, proximity, top-K and champion lists; the Dhvani key and pooled df; dense retrieval, script-consistency distillation, vocabulary pruning, cluster pruning; learning to rank; the gated cascade; the evaluation framework and metrics; the demo; the report. **Anamika Pal:** human query annotation and the annotator-agreement analysis. **Abhinav Bachchas:** human query annotation, final evaluation runs and plots, proofreading. *[Update to what actually happened before submitting.]*

AI-use declaration

We used Claude Code (an AI coding assistant by Anthropic) to help brainstorm and plan the project, write and refactor code, write tests and draft documentation. Devansh chose the track and the final idea, directed the work, reviewed the code and checked the results. The human query annotations were written without AI tools. Every number in this report was produced by the scripts in the repository on the real data.

References

1. X. Zhang et al. MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages. TACL, 2023.
2. P. Gupta, K. Bali, R. E. Banchs, M. Choudhury, P. Rosso. Query Expansion for Mixed-Script Information Retrieval. SIGIR, 2014.
3. R. Sequiera et al. Overview of FIRE-2015 Shared Task on Mixed Script Information Retrieval. FIRE, 2015.
4. A. Pirkola. The Effects of Query Structure and Dictionary Setups in Dictionary-Based Cross-Language Information Retrieval. SIGIR, 1998.
5. W. Webber, A. Moffat, J. Zobel. A Similarity Measure for Indefinite Rankings. ACM TOIS, 2010.
6. S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. FNTIR, 2009.
7. G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR, 2009.
8. A. Shtok, O. Kurland, D. Carmel, F. Raiber, G. Markovits. Predicting Query Performance by Query-Drift Estimation. ACM TOIS, 2012.
9. L. Wang, J. Lin, D. Metzler. A Cascade Ranking Model for Efficient Ranked Retrieval. SIGIR, 2011.
10. T.-Y. Liu. Learning to Rank for Information Retrieval. FNTIR, 2009.
11. N. Reimers, I. Gurevych. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. EMNLP, 2020.
12. A. Abdaoui, C. Pradel, G. Sigel. Load What You Need: Smaller Versions of Multilingual BERT. SustainLP, 2020.
13. A. Ramanathan, D. D. Rao. A Lightweight Stemmer for Hindi. EACL Workshop on Computational Linguistics for South Asian Languages, 2003.
14. L. Wang et al. Multilingual E5 Text Embeddings: A Technical Report. arXiv, 2024.
15. C. D. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.

Data: MIRACL (Apache-2.0), passage text from Wikipedia (CC BY-SA 3.0). Model: intfloat/multilingual-e5-small (MIT).

Appendix

A. Query latency per system (laptop CPU, Intel i5-12450H, no GPU)

	system	median ms	p95 ms
B0	BM25, raw whitespace tokens	0.9	26.8
B1	BM25 + normalisation + stemming	0.7	9.3
B1N	B1 without stemming	0.8	7.2
B2	B1 + Roman→Devanagari transliteration	2.8	11.5
V1	tf-idf Inc.Ltc cosine	0.7	9.4
S0	B1 + classic Soundex zone	11.3	20.1
S1	B1 + Dhvani zone	5.4	12.3
S2	S1 + pooled df	7.0	16.5
S3	S2 + title zone + proximity	14.0	29.7
D0	dense e5-small (int8)	12.5	15.7
D1	dense, SCD query encoder	12.5	16.3
H1	RRF(S3, D1)	23.8	38.7
L1	learning to rank	35.5	60.8
G1	gated cascade	18.6	52.8

B. Learning-to-rank weights

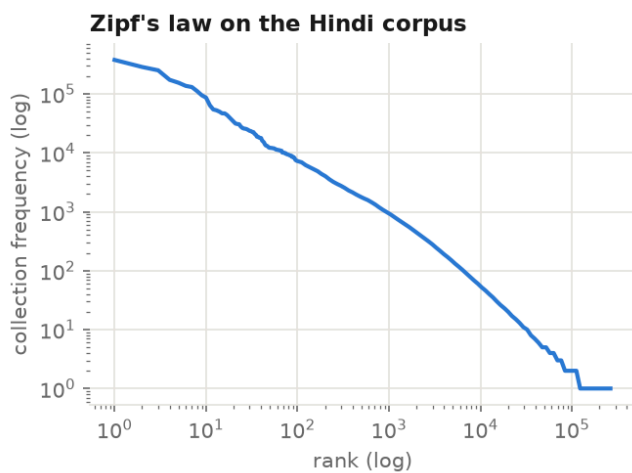
LTR feature	weight
bm25_all	-0.069
bm25_dhvani	+0.904
bm25_title	-0.189
proximity	+0.172
tfidf	+0.273
dense	+1.363
sparse_rr	+0.619
dense_rr	+0.392
lead_passage	+0.021
log_length	+0.113
roman_query	-0.226

C. Gate weights

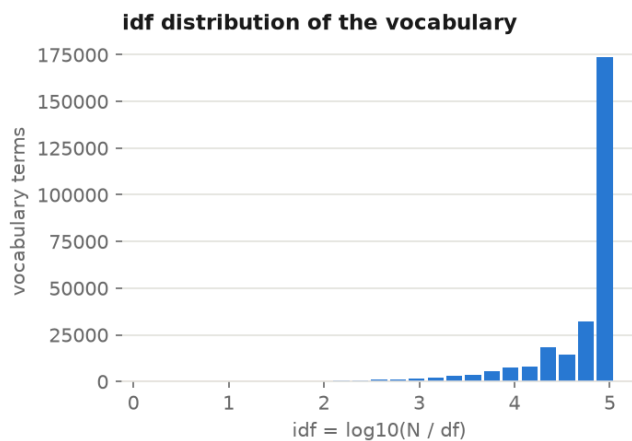
gate feature	weight
top_score	-0.038
score_gap	-0.315
nqc	+0.141
dhvani_only	-0.403
unmatched	-0.090
max_idf	+0.083
query_length	-0.165
roman_query	-0.079

D. Significance (paired randomisation test, 10,000 permutations, nDCG@10)

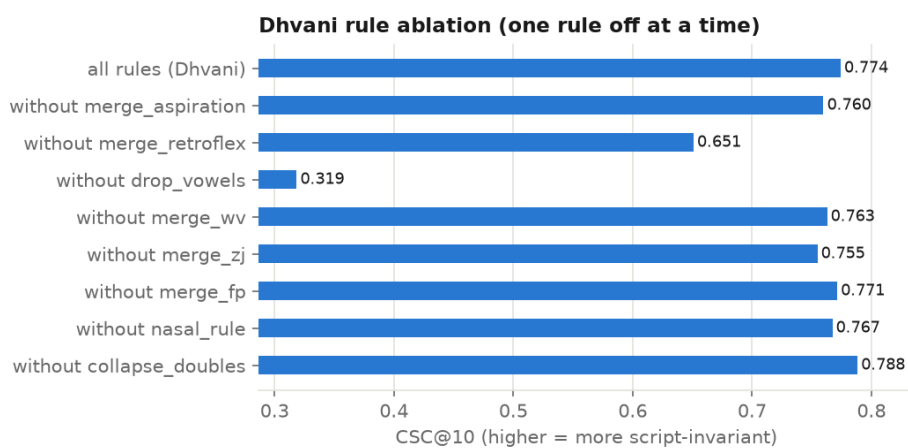
comparison	form	nDCG A	nDCG B	difference	p-value	p < 0.05
S1 vs B1	F1	0.544	0.533	+0.011	0.2645	no
S1 vs B1	F2	0.481	0.004	+0.477	0.0001	yes
S1 vs B1	F3	0.488	0.003	+0.485	0.0001	yes
S1 vs S0	F1	0.544	0.518	+0.026	0.0033	yes
S1 vs S0	F2	0.481	0.344	+0.137	0.0001	yes
S1 vs S0	F3	0.488	0.320	+0.168	0.0001	yes
S3 vs B2	F1	0.543	0.531	+0.012	0.3246	no
S3 vs B2	F2	0.488	0.113	+0.376	0.0001	yes
S3 vs B2	F3	0.491	0.095	+0.396	0.0001	yes
S2 vs S1	F1	0.533	0.544	-0.011	0.0106	yes
S2 vs S1	F2	0.487	0.481	+0.006	0.0082	yes
S2 vs S1	F3	0.490	0.488	+0.002	0.1650	no
S3 vs S2	F1	0.543	0.533	+0.009	0.2235	no
S3 vs S2	F2	0.488	0.487	+0.001	0.7658	no
S3 vs S2	F3	0.491	0.490	+0.001	0.7812	no
D1 vs D0	F1	0.598	0.601	-0.003	0.6915	no
D1 vs D0	F2	0.167	0.001	+0.166	0.0001	yes
D1 vs D0	F3	0.151	0.000	+0.151	0.0001	yes
H1 vs S3	F1	0.636	0.543	+0.093	0.0001	yes
H1 vs S3	F2	0.427	0.488	-0.061	0.0002	yes
H1 vs S3	F3	0.426	0.491	-0.065	0.0001	yes
L1 vs H1	F1	0.642	0.636	+0.006	0.5075	no
L1 vs H1	F2	0.501	0.427	+0.074	0.0001	yes
L1 vs H1	F3	0.498	0.426	+0.072	0.0001	yes
G1 vs H1	F1	0.633	0.636	-0.003	0.4160	no
G1 vs H1	F2	0.492	0.427	+0.065	0.0001	yes
G1 vs H1	F3	0.490	0.426	+0.064	0.0001	yes
B1 vs B1N	F1	0.533	0.492	+0.041	0.0003	yes
B1 vs B1N	F2	0.004	0.004	+0.000	1.0000	no
B1 vs B1N	F3	0.003	0.003	+0.000	1.0000	no



Zipf plot of the corpus.



idf distribution of the vocabulary.



Dhvani rule ablation (CSC@10).